



## Heterogeneous Computing, Übungsblatt 2, Sommer 2026

### Aufgabe 1: Das SIMT-Ausführungsmodell und Warp-Divergenz (rechenintensiv)

Ziel: sichtbar machen, dass eine GPU sehr viele Threads im Gleichschritt (SIMT) abarbeitet und dass datenabhängige Verzweigungen innerhalb eines Warps teuer werden, während sie eine CPU kaum stören. Implementieren Sie dazu einen geeigneten, rechenintensiven Kernel mit minimalem Speicherbedarf: jeder Thread bearbeitet genau ein Element und führt darauf eine konfigurierbare, rein arithmetische Last aus (z. B. eine iterierte Polynom-/Newton-Berechnung oder ein synthetisches „ALU-Mischen“ über  $k$  Iterationen – ohne nennenswerte Speicherzugriffe). Skalieren Sie die Problemgröße  $n$  bis in die Millionen. Messen Sie den Durchsatz Ihres Kernels (z. B. Operationen/s bzw. GFLOP/s) und stellen Sie die Skalierung über  $n$  dar. Halten Sie fest, ab welcher Problemgröße die GPU „warmläuft“ und wie viele Threads Sie starten müssen, um das Gerät auszulasten – ein Aufwand, der bei einer CPU-Lösung in dieser Form gar nicht erst anfällt. Führen Sie nun gezielt Warp-Divergenz ein: lassen Sie den Arbeitsaufwand pro Thread datenabhängig variieren (z. B. Verzweigung über `threadIdx % k` oder eine vom Element abhängige Schleifenlänge). Variieren Sie den Divergenzgrad systematisch von „keine Divergenz“ bis „jeder Thread im Warp nimmt einen anderen Pfad“ und messen Sie den Einbruch bei der Gesamtperformanz. Beschäftigen Sie sich mit der Frage: Warum serialisiert ein Warp divergente Pfade, und warum ist dieselbe Verzweigung auf der CPU (Sprungvorhersage, unabhängige Kerne) nahezu kostenlos? Genau hier wird der SIMT-Unterschied zu einer „normalen“ CPU sichtbar.

### Aufgabe 2: Speicherzugriff: Latenz verstecken und Bandbreite (speicherintensiv)

Ziel: sichtbar machen, dass die GPU Speicherlatenz nicht durch große Caches sondern durch sehr viele gleichzeitig lauffähige Warps versteckt – und dass das Zugriffsmuster über die tatsächlich erreichte Bandbreite entscheidet. Implementieren Sie einen speicherintensiven Kernel mit geringer arithmetischer Intensität (z. B. Streaming à la  $b[i] = a[i] * c$ ). Vergleichen Sie die folgenden drei Zugriffsmuster auf den Gerätespeicher: (1) zusammenhängend / coalesced (Stride 1), (2) gestriped (Stride  $k$ ) und (3) zufällig (Gather). Messen Sie für jedes Muster die erreichte effektive Bandbreite (GB/s) und setzen Sie sie ins Verhältnis zur maximal erreichbaren Bandbreite des Geräts (über `deviceQuery` o.ä. abfragbar). Zeigen Sie das Verstecken der Latenz über die Occupancy: variieren Sie die Blockgröße bzw. die Anzahl gleichzeitig residenter Warps und zeigen Sie, wie der Durchsatz davon abhängt, dass genügend Warps bereitstehen, um Speicher-Stalls zu überbrücken. Geben Sie die gemessene Occupancy an (Occupancy = aktive Warps / maximal mögliche aktive Warps). Erklären Sie den Unterschied der beiden Latenz-Strategien: Die GPU versteckt Latenz über massive Parallelität (viele Warps), während die CPU Latenzen über die Cache-Hierarchie und Prefetching verbirgt (sequenziell schnell, zufällig durch Cache-Misses langsam). Welche Konsequenzen hat dieser grundsätzlich andere Mechanismus im Hinblick auf eine effiziente Anordnung von Daten in GPU-Anwendungen?

### Abgabe

Abzugeben ist ein Repository mit Ihrer Implementierung als Pull-Request gegen das Github-Repository [2026S-HC](#). In diesem Repository soll sich auch ein kurzer Ergebnisbericht befinden. Vermeiden Sie jegliche personenbezogenen Informationen (also keine Matrikelnummer, kein Name), damit am Ende der Vorlesung alle Lösungen in diesem Repository zusammengefasst und veröffentlicht werden können. Legen Sie innerhalb des Repositories einen eigenen Unterordner an, in dem sich alle Lösungen zu diesem Aufgabenblatt inkl. dem Bericht befinden; das macht das Zusammenführen aller Abgaben etwas leichter. Außerdem müssen Sie in der Übung Ihre Realisierung in einem ca. 10-minütigen Vortrag vorstellen.

# 1 Aufgabe 1

Die Messungen wurden auf einer NVIDIA GeForce RTX 4060 durchgeführt. Die verwendete GPU besitzt 24 SMs, 3072 FP32-Kerne, einen FP32-Peak von ungefähr 15,3 TFLOP/s und eine Speicherbandbreite von 272 GB/s.

## 1.1 Verwendete Kernel

### 1.1.1 FMA-Kernel für Skalierung und GPU-Warm-up

Der erste verwendete Kernel ist ein *fused multiply add (fma)* Kernel: `kernel_fma` in `fma.cu`. Der Kernel simuliert eine große arithmetische Last bei gleichzeitig wenigen Speicherzugriffen. Damit kann gemessen werden, ab welcher Problemgröße genügend Threads vorhanden sind, um die FP32-Recheneinheiten der GPU voll auszulasten.

Jeder Thread bearbeitet genau ein Element, d.h. er lädt einen 4 Byte großen Wert, führt mindestens eine FMA Operation auf diesem Wert aus und schreibt anschließend das Ergebnis zurück. Dadurch ist der Speicherbedarf minimal (4 Byte *read* und 4 Byte *write*), während die arithmetische Last über die Anzahl der Iterationen der FMA Operation mithilfe des Parameters `iters` skaliert werden kann.

```
float x = in[idx];
float acc0 = x, acc1 = x * 0.5f + 1.0f;

for (int i = 0; i < iters; ++i) {
    acc0 = fmaf(acc0, a, b);
    acc1 = fmaf(acc1, a, b);
}

out[idx] = acc0 + acc1;
```

Das Ergebnis wird in den Ausgabepuffer (`out[idx]`) geschrieben, damit der Compiler die Rechenschleife nicht entfernen kann.

### 1.1.2 FMA Divergenz-Kernel für unterschiedliche Kontrollflusspfade

Der zweite verwendete Kernel ist ebenfalls ein *fused multiply add (fma)* Kernel: `kernel_fma_div` in `fma.cu`. Im Unterschied zum 1. Kernel fügt dieser gezielt einen datenabhängigen Kontrollfluss hinzu. Damit kann nicht nur die reine Rechenleistung gemessen werden, sondern auch der Effekt, der entsteht, wenn verschiedene *Lanes* eines Warps unterschiedliche Pfade nehmen können.

Im Unterschied zu `kernel_fma` ist die Arbeit pro Thread in `kernel_fma_div` nicht uniform verteilt. Einige Threads nehmen einen schweren Pfad mit vielen FMA-Operationen, während andere einen leichten Pfad mit deutlich weniger Arbeit nehmen. Der Parameter `stride` steuert dabei, ob diese Pfade innerhalb eines Warps gemischt sind oder ob ganze Warps homogen bleiben.

### 1.1.3 Mandelbrot Kernel der das natürliche Auftreten von Divergenz simuliert

Der Kernel in `mandelbrot.cu` berechnet für jeden Pixel eine Mandelbrot-Iteration

```
z = z * z + c
```

bis entweder  $|z| > 2$  gilt oder `maxIter` erreicht ist. Es handelt sich hierbei ebenfalls um ein rechenintensives Problem mit sehr wenigen Speicherzugriffen, da jeder Thread am Ende im Wesentlichen nur die erreichte Iterationszahl schreibt.

Der Unterschied zum FMA-Kernel ist, dass die Divergenz nicht künstlich über `stride` eingestellt wird. Sie entsteht natürlich aus den Daten: Pixel außerhalb der Mandelbrot-Menge brechen früh ab, Pixel nahe am Rand laufen deutlich länger, und Pixel im Inneren erreichen oft `maxIter`. Wenn solche Pixel in derselben Warp liegen, müssen die unterschiedlich langen Schleifenpfade serialisiert werden. Mandelbrot zeigt also denselben SIMT-Effekt in einem realistischeren Algorithmus als der FMA-Kernel, ist aber weniger exakt kontrollierbar als `kernel_fma_div`.

#### 1.1.4 Monte Carlo Kernel um das zufällige Auftreten von Divergenz zu simulieren

Der Monte-Carlo Kernel in `montecarlo.cu` schätzt  $\pi$ , indem jeder Thread zufällige Punkte im Einheitsquadrat erzeugt und prüft, ob sie im Viertelkreis liegen:

```
if (x * x + y * y <= 1.0f)
    ++local;
```

Dabei ist die Divergenz nicht räumlich strukturiert wie bei der Berechnung der Mandelbrot Menge, sondern sie entsteht zufällig durch die Stichproben. In einer Warp nehmen die Lanes daher statistisch gemischte Pfade.

Monte Carlo ist allerdings kein isolierter Divergenz-Benchmark, da Die Laufzeit zusätzlich Kosten für die Erzeugung der Zufallszahlen, lokale Zähler und eine globale Reduktion per `atomicAdd` enthält. Deshalb dient dieser Kernel nur als ergänzender Referenzfall, um zu illustrieren, wie zufällige Kontrollflüsse in einem stochastischen GPU-Algorithmus auftreten.

## 1.2 Skalierung über die Problemgröße

Für die Skalierung wurde die Anzahl der Iterationen fest gewählt: `iters = 512`. Die Problemgröße  $n$  wurde von 1024 bis 134 Millionen Elementen variiert. Die Ergebnisse liegen in `outputs/scaling.csv`; die daraus erzeugte Grafik ist in Abbildung 1 dargestellt.

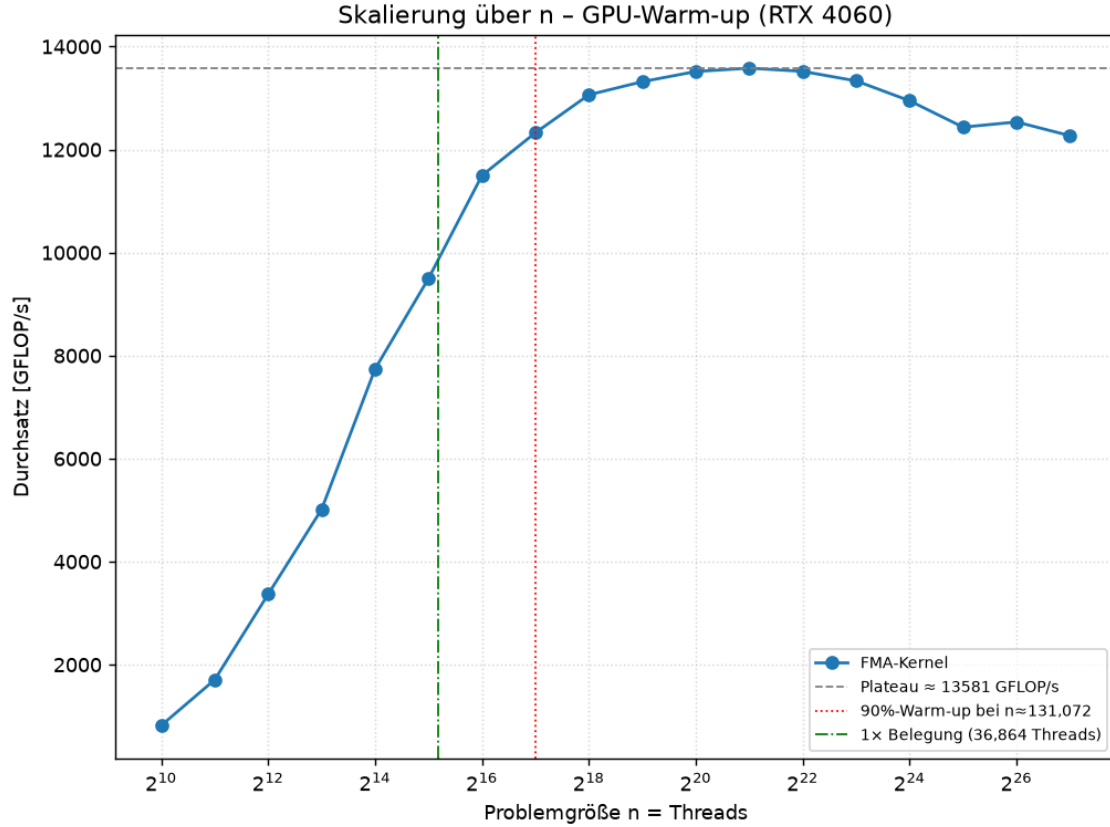


Abbildung 1: Durchsatz des FMA-Kernels in Abhängigkeit von der Problemgröße.

| n = Threads | GFLOP/s | Vielfaches der Sättigung |
|-------------|---------|--------------------------|
| 1 024       | 819     | 0,03                     |
| 16 384      | 7 740   | 0,44                     |
| 32 768      | 9 498   | 0,89                     |
| 65 536      | 11 498  | 1,78                     |
| 131 072     | 12 327  | 3,6                      |
| 262 144     | 13 064  | 7,1                      |
| 1 048 576   | 13 515  | 28                       |
| 16 777 216  | 12 947  | 455                      |
| 67 108 864  | 12 536  | 1820                     |

Tabelle 1: Auszug aus der Skalierungsmessung.

Die theoretische Sättigungsschwelle der verwendeten GPU liegt somit bei

$$24 \text{ SMs} \cdot 1536 \text{ Threads/SM} = 36\,864$$

gleichzeitig laufenden Threads. Diese Schwelle ist jedoch nicht identisch mit dem praktischen Plateau. In der Messung wurde die 90-Prozent-Region erst bei etwa 131 072 Threads erreicht, also bei ungefähr dem 3,6-fachen der Thread-Kapazität. Das Plateau liegt bei etwa 13,6 TFLOP/s.

Die Messung zeigt damit den typischen GPU-Warm-up: Bei kleinen Problemgrößen ist die GPU nicht ausgelastet. Erst wenn die Problemgröße groß genug ist, d.i. ausreichend viele Warps und genügend Arbeit vorhanden ist, kann der Scheduler Latenzen durch Wechseln auf andere lauffähige Warps ausgleichen. Dagegen sind die Kerne einer CPU auf niedrige Latenz, Sprungvorhersage, Out-of-Order-Ausführung und große Caches optimiert, sodaß ein einzelner Thread

bereits sinnvoll Arbeit leisten kann. Im Unterschied zur CPU muss die GPU mit sehr vielen unabhängigen Threads versorgt werden.

### 1.3 Gezielte Warp-Divergenz

Auch für den Divergenzversuch mit `kernel_fma_div` hat ein Thread genau ein Element bearbeitet, wobei die Rechenlast dieses Mal von der Thread-ID abhing. Dabei hat die Hälfte der Threads einen schweren Pfad mit `iters` FMA Operationen, die andere Hälfte einen leichten Pfad mit `iters/8` FMA Operationen genommen. Der Parameter `stride` hat festgelegt, wie diese Pfade auf die Lanes eines Warps verteilt wurden.

```
bool heavy = ((idx / stride) & 1) == 0;

if (heavy) {
    for (int i = 0; i < iters; ++i)
        acc = fmaf(acc, a, b);
} else {
    for (int i = 0; i < iters / 8; ++i)
        acc = fmaf(acc, a, b);
}
```

Während sich bei `stride = 1` schwere und leichte Threads innerhalb eines Warps maximal stark abwechseln, sind Warps bei *strides*, die Vielfache von 32 sind, homogen. Strides knapp oberhalb von 32 schneiden dagegen wieder durch Warp-Grenzen und erzeugen teilweise divergente Warps. Dadurch kann direkt gemessen werden, ob der Performance-Einbruch wirklich ein Intra-Warp-Effekt ist.

Die Messdaten liegen in `outputs/divergence.csv`; die daraus erzeugte Grafik ist in Abbildung 2 dargestellt.

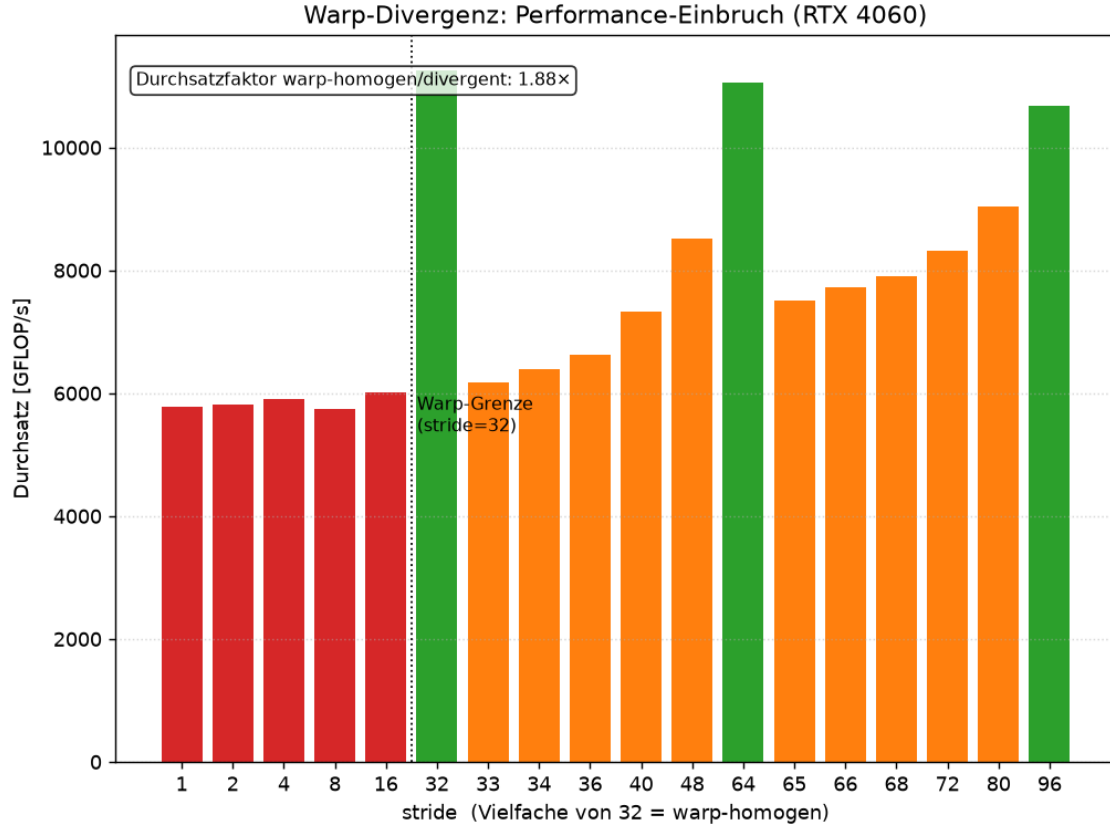


Abbildung 2: Performance-Einbruch durch Warp-Divergenz.

| Stride | GFLOP/s | Zeit [ms] | Regime          |
|--------|---------|-----------|-----------------|
| 1      | 5 778   | 13,38     | max. divergent  |
| 2      | 5 822   | 13,28     | divergent       |
| 4      | 5 913   | 13,08     | divergent       |
| 8      | 5 739   | 13,47     | divergent       |
| 16     | 6 016   | 12,85     | divergent       |
| 32     | 11 264  | 6,86      | divergenzfrei   |
| 33     | 6 170   | 12,53     | teils divergent |
| 34     | 6 384   | 12,11     | teils divergent |
| 36     | 6 620   | 11,68     | teils divergent |
| 40     | 7 334   | 10,54     | teils divergent |
| 48     | 8 525   | 9,07      | teils divergent |
| 64     | 11 065  | 6,99      | divergenzfrei   |
| 65     | 7 511   | 10,29     | teils divergent |
| 66     | 7 732   | 10,00     | teils divergent |
| 68     | 7 901   | 9,78      | teils divergent |
| 72     | 8 315   | 9,30      | teils divergent |
| 80     | 9 039   | 8,55      | teils divergent |
| 96     | 10 684  | 7,24      | divergenzfrei   |

Tabelle 2: Divergenzmessung bei gleicher Gesamtarbeit.

Die Auswertung zeigt, dass der Durchsatz genau bei *strides* springt, die Vielfache der Warp-Größe 32 sind. Für *stride* < 32 enthält jeder Warp beide Kontrollflusspfade, sodaß der Warp diese Pfade serialisieren muss. Der gemessene Durchsatz liegt in diesem Fall nur bei etwa 5,7

bis 6,0 TFLOP/s. Bei `stride = 32, 64` und `96` sind die Warps homogen; der Durchsatz steigt auf etwa 10,7 bis 11,3 TFLOP/s. Die Zwischenwerte `33` bis `48` und `65` bis `80` schneiden die Warp-Grenzen nicht sauber und sind deshalb nur teilweise divergent. Die vollständig divergente Variante ist gegenüber den warp-homogenen Fällen ungefähr um den Faktor 1,9 langsamer.

## 1.4 Warum Divergenz auf der GPU teuer ist

Eine GPU führt Threads in Warps aus, die in der getesteten GPU aus 32 Lanes bestehen. Im SIMT-Modell müssen alle Lanes gemeinsam einen Instruktionsstrom ausführen. Das heißt, wenn einzelne Lanes unterschiedliche Pfade eines `if/else` nehmen, kann die GPU diese Pfade nicht gleichzeitig als unabhängige Kontrollflüsse ausführen. Stattdessen wird zuerst einer der beiden Pfade mit einer Aktivitätsmaske ausgeführt, danach der andere Pfad mit der umgekehrten Maske. Lanes, die zum jeweils anderen Pfad gehören, sind in dieser Zeit inaktiv.

Dadurch addieren sich die Laufzeiten der Pfade innerhalb des Warps. Im Extremfall muss ein Warp viele verschiedene Pfade nacheinander ausführen. Genau dieser Effekt ist in Abbildung 2 sichtbar: Der Performance-Sprung liegt nicht bei beliebigen Blockgrößen, sondern exakt an den Vielfachen der Warp-Grenze von 32 Threads.

## 1.5 Warum dieselbe Verzweigung auf der CPU weniger stört

Eine CPU besitzt kein Warp-Modell mit 32 aneinander gekoppelten Lanes. Stattdessen führt ein CPU-Kern einen einzelnen Instruktionsstrom aus und reagiert auf Verzweigungen mit Sprungvorhersage und spekulativer Ausführung. Bei gut vorhersagbaren Mustern kostet eine Verzweigung daher fast nichts. Zudem besitzt jeder CPU-Kern jeweils einen eigenen Kontrollfluss.

Der Architekturunterschied ist deshalb grundsätzlich: Die GPU spart Steuerlogik und erzielt bei einem uniformen Kontrollfluss einen sehr hohen Durchsatz, muss dafür aber bei einem divergenten Kontrollfluss innerhalb eines Warps serialisieren. Dagegen investiert die CPU wesentlich mehr Logik pro Kern, um auch einen komplexen Kontrollfluss mit niedriger Latenz auszuführen, kann dafür allerdings nicht den Durchsatz einer GPU bei einem uniformen Kontrollfluss erreichen.

## 1.6 Anhang: Ergänzende Referenzkernel

Zusätzlich zu den synthetischen FMA- und Divergenz-Kernen wurden der Mandelbrot- und der Monte-Carlo-Kernel als ergänzende Referenzfälle ausgewertet.

### 1.6.1 Mandelbrot-Kernel

Der Mandelbrot-Kernel berechnet für jeden Pixel, nach wie vielen Iterationen die Folge  $z_{n+1} = z_n^2 + c$  den Radius 2 überschreitet. Dadurch hat jeder Thread eine datenabhängige Laufzeit. Punkte außerhalb der Menge brechen früh ab, Punkte nahe am Rand oder innerhalb der Menge laufen deutlich länger.

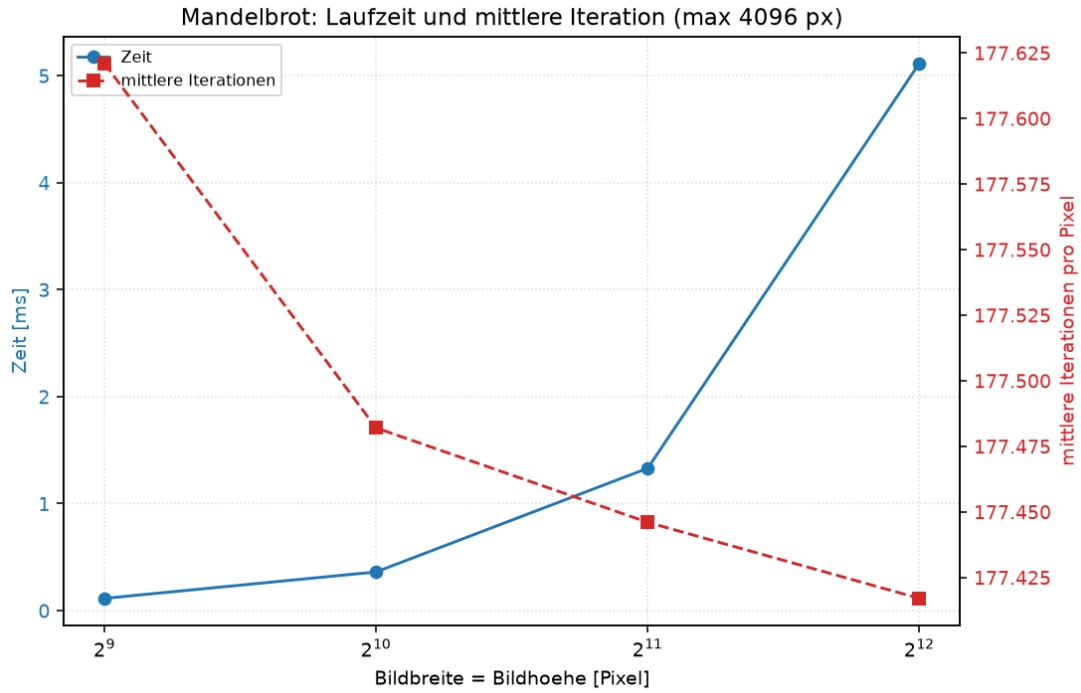


Abbildung 3: Mandelbrot-Sweep: Laufzeit und mittlere Iterationszahl pro Pixel.

| Auflösung | Pixel      | Zeit [ms] | GFLOP/s,<br>obere<br>Schranke | mittl. Iter. |
|-----------|------------|-----------|-------------------------------|--------------|
| $512^2$   | 262 144    | 0,114     | 22 939                        | 177,6        |
| $1024^2$  | 1 048 576  | 0,361     | 29 008                        | 177,5        |
| $2048^2$  | 4 194 304  | 1,330     | 31 527                        | 177,4        |
| $4096^2$  | 16 777 216 | 5,115     | 32 797                        | 177,4        |

Tabelle 3: Mandelbrot-Messung mit `maxIter` = 1000. Die GFLOP/s-Spalte ist eine obere Schranke, da sie für jedes Pixel die maximale Iterationszahl annimmt.



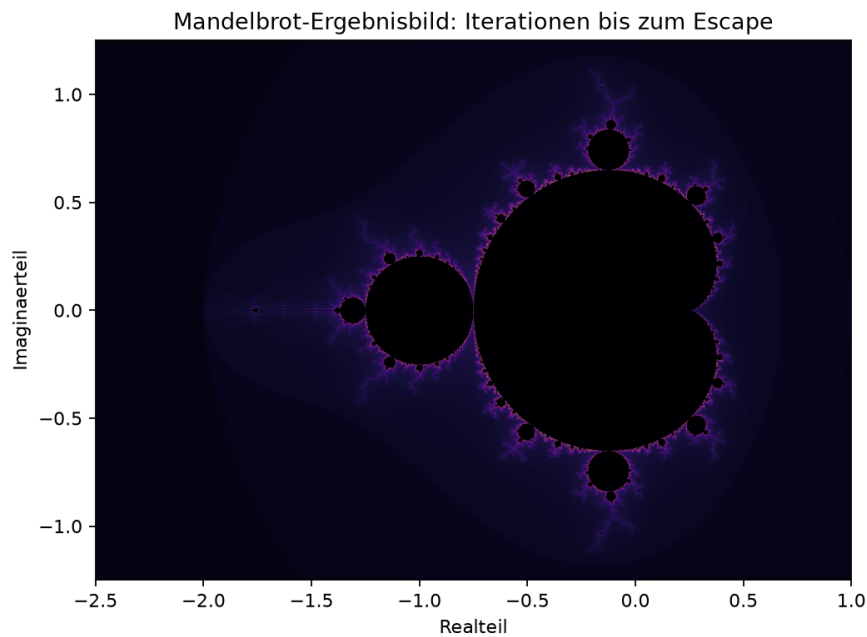


Abbildung 4: Mandelbrot: Die Farbe kodiert die Anzahl der Iterationen bis zum Abbruch.

Die Messung zeigt, dass die mittlere Iterationszahl über die skalierten Auflösungen nahezu konstant bleibt, während die Laufzeit mit der Pixelzahl wächst. Das Bild illustriert die Ursache der Divergenz sichtbar: benachbarte Pixel können sehr unterschiedliche Iterationszahlen besitzen, insbesondere am Rand der Menge.

### 1.6.2 Monte-Carlo-Kernel

Der Monte-Carlo-Kernel schätzt  $\pi$ , indem zufällige Punkte im Einheitsquadrat geraten werden. Ein Punkt zählt als Treffer, wenn  $x^2 + y^2 \leq 1$  gilt. Die Divergenz hängt in diesem Fall direkt von den zufällig gezogenen Zahlen ab.

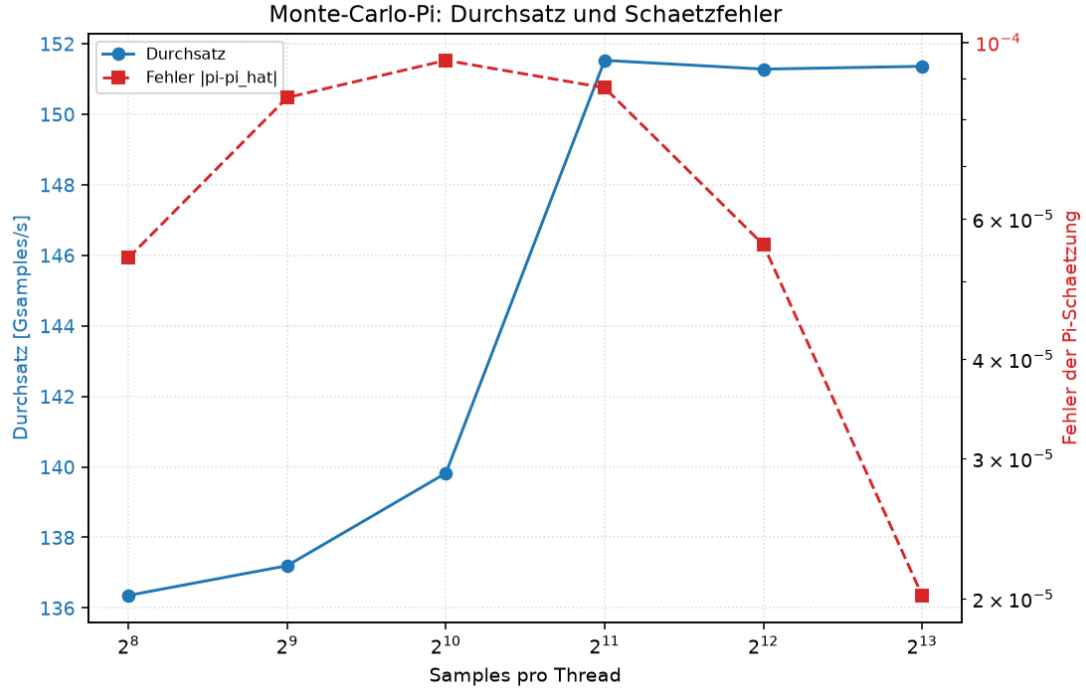


Abbildung 5: Monte-Carlo-Sweep: Stichprobendurchsatz und Fehler der  $\pi$ -Schätzung.

| Samp./Th. | Samples<br>[Mrd.] | Zeit [ms] | Gsamp./s | $\hat{\pi}$ | Fehler               |
|-----------|-------------------|-----------|----------|-------------|----------------------|
| 256       | 0,268             | 1,969     | 136,36   | 3,141646    | $5,36 \cdot 10^{-5}$ |
| 512       | 0,537             | 3,913     | 137,20   | 3,141678    | $8,52 \cdot 10^{-5}$ |
| 1024      | 1,074             | 7,679     | 139,82   | 3,141688    | $9,49 \cdot 10^{-5}$ |
| 2048      | 2,147             | 14,172    | 151,53   | 3,141680    | $8,76 \cdot 10^{-5}$ |
| 4096      | 4,295             | 28,391    | 151,28   | 3,141648    | $5,57 \cdot 10^{-5}$ |
| 8192      | 8,590             | 56,752    | 151,36   | 3,141613    | $2,02 \cdot 10^{-5}$ |

Tabelle 4: Monte-Carlo-Messung mit  $2^{20}$  Threads.

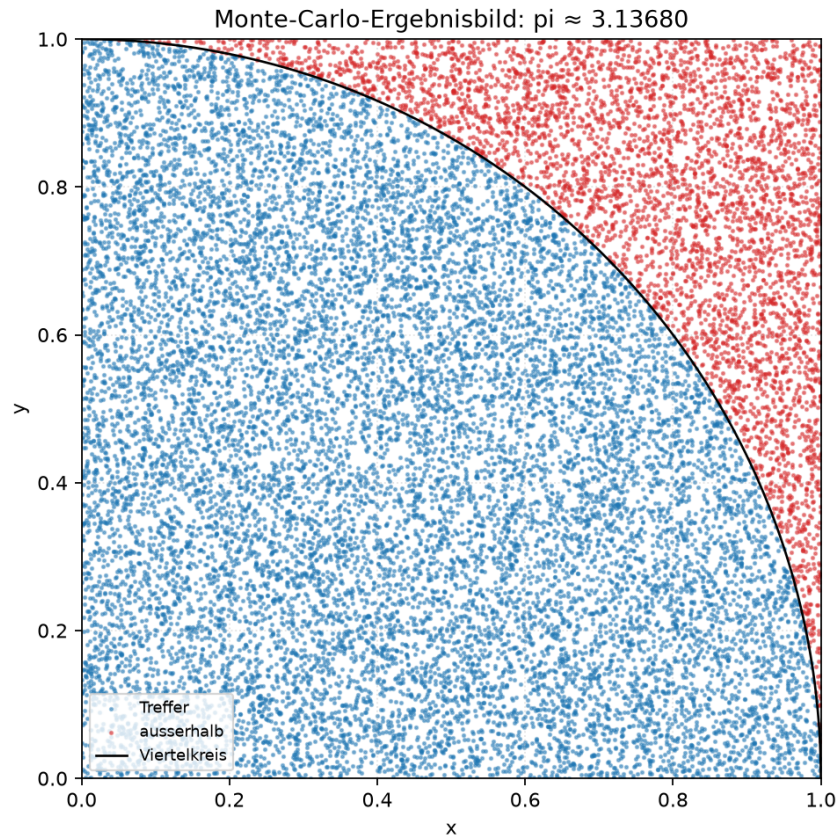


Abbildung 6: Monte-Carlo-Ergebnisbild: zufällige Punkte im Einheitsquadrat, getrennt nach Treffern innerhalb des Viertelkreises und Punkten außerhalb.

Die Tabelle zeigt, dass der Durchsatz ab größeren Stichprobenzahlen pro Thread bei etwa 151 Gsamples/s liegt. Der Fehler der  $\pi$ -Schätzung sinkt nicht strikt monoton, da die Messung stochastisch ist; mit wachsender Gesamtzahl an Samples wird die Schätzung aber tendenziell stabiler.

## 2 Aufgabe 2

Die Messungen wurden auf einer NVIDIA GeForce RTX 4060 durchgeführt. Laut Geräteabfrage liegt die theoretische Speicherbandbreite bei 272 GB/s. Die GPU besitzt 24 SMs und maximal 1536 Threads pro SM, also 48 Warps pro SM.

### 2.1 Speicherintensiver Kernel

Ein speicherintensiver Kernel wurde in `memory.cu` implementiert. Jeder Thread liest einen Wert aus `a` multipliziert diesen mit einer Konstanten `c` und schreibt diesen Wert nach `b`.

```
b[i] = a[i] * c;
```

Während der Rechenaufwand pro Element mit höchstens 0,25 FLOP/Byte sehr gering ist, entstehen Speicherzugriffe im Umfang von 8 Byte durch einen 4 Byte *read* und einen 4 Byte *write*. Damit wird die Laufzeit des Kernels primär durch das Speicherzugriffsmuster und nicht die ALUs bestimmt.

Es wurden insgesamt drei Varianten des Kernels implementiert:

- `kernel_copy_coalesced`: benachbarte Threads lesen benachbarte Adressen.
- `kernel_copy_strided`: Thread `i` liest `a[(i * stride) % n]`.
- `kernel_copy_gather`: Thread `i` liest eine zufällige Adresse `a[idx[i]]`.

#### 2.1.1 STREAM-Triad als Referenzfall

Zusätzlich wurde ein STREAM-Kernel in `stream.cu` als Referenz implementiert. Dieser bearbeitet ein klassisches Bandbreitenproblem:

```
a[i] = b[i] + alpha * c[i];
```

Pro Element werden zwei Werte `b[i]` und `c[i]` gelesen und ein Wert `a[i]` geschrieben. Das erzeugt bei FP32 Speicherzugriffe im Umfang von 12 Bytes für nur eine *fused multiply add (fma)*. Diese hat einen arithmetischen Aufwand von etwa 0,17 FLOP/Byte.

Der Unterschied zu `memory.cu` besteht darin, dass

- `stream.cu` einen einzelnen, gutartigen STREAM-Triad-Fall misst. Bei dem alle Threads regulär und zusammenhängend auf `a`, `b` und `c` zugreifen. Der Kernel beantwortet damit die Frage: Welche Bandbreite ist bei einem sauberen Streaming-Zugriff erreichbar?
- `memory.cu` variiert dagegen gezielt das Zugriffsmuster. Derselbe einfache Rechenkern wird als *coalesced*, *strided* und *random gather* ausgeführt. Damit beantwortet der Kernel die Frage: Wie stark bricht die Bandbreite ein, wenn die Adressen innerhalb einer Warps ungünstig liegen?

#### 2.1.2 Zugriffsmuster und effektive Bandbreite

Für den Sweep wurden 33 554 432 Elemente verwendet. Die Messwerte liegen in `patterns.csv`; die daraus erzeugte Grafik ist in Abbildung 7 dargestellt.

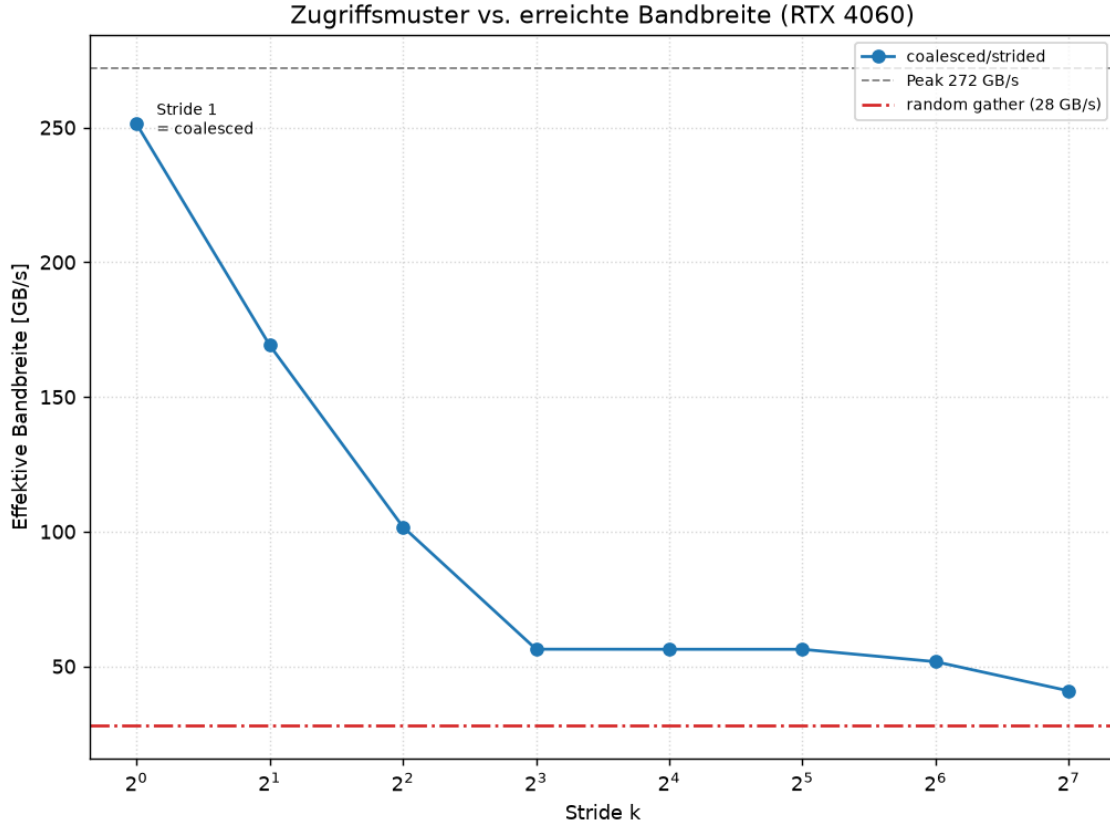


Abbildung 7: Effektive Bandbreite für coalesced, strided und Gather-Zugriffe.

| Muster        | Stride | GB/s   | % Peak |
|---------------|--------|--------|--------|
| coalesced     | 1      | 251,30 | 92,38  |
| strided       | 2      | 169,15 | 62,18  |
| strided       | 4      | 101,76 | 37,41  |
| strided       | 8      | 56,34  | 20,71  |
| strided       | 16     | 56,31  | 20,70  |
| strided       | 32     | 56,30  | 20,70  |
| strided       | 64     | 51,69  | 19,00  |
| strided       | 128    | 40,87  | 15,02  |
| random gather | -      | 28,03  | 10,30  |

Tabelle 5: Gemessene effektive Bandbreite für verschiedene Zugriffsmuster.

Der *coalesced* Fall (*stride*= 1) erreicht mit 251,30 GB/s etwa 92 Prozent des theoretischen Peaks. Das ist der Idealfall für eine GPU: Alle 32 Lanes eines Warps lesen zusammenhängende Adressen, sodass die Hardware wenige große Speichertransaktionen bilden kann. Damit ist fast jedes übertragene Byte nutzbar.

Bei *strided* Zugriffen (*stride*> 1) sinkt die nutzbare Bandbreite dagegen deutlich. Schon bei einem *stride* von 2 fällt die nutzbare Bandbreite auf 169,15 GB/s und für einen *stride* von 4 auf 101,76 GB/s. Ab einem *stride* von 8 werden nur noch etwa 20 Prozent des Peaks ( $\approx 50$  GB/s) erreicht. Das liegt daran, dass mit zunehmendem *stride* die Lanes eines Warps weiter auseinanderliegende Adressen lesen müssen. Dadurch müssen mehr Speichersegmente geladen werden, obwohl pro Segment nur ein kleiner Teil tatsächlich genutzt wird.

*Random Gather* hat mit 28,03 GB/s die geringste nutzbare Bandbreite. Jede Lane liest praktisch eine zufällige Adresse, wodurch *coalescing* und *räumliche Lokalität* fast vollständig

verloren gehen. Viele einzelne Speichertransaktionen und Cache-Misses senken dabei die effektive Bandbreite auf etwa 10 Prozent des Peaks.

### 3 *Occupancy* und *latency hiding*

Im zweiten Experiment wurde die Blockgröße des *coalesced* Kernels variiert. Für jede Blockgröße wurde die theoretische *occupancy* mit `cudaOccupancyMaxActiveBlocksPerMultiprocessor()` bestimmt:

$$\text{Occupancy} = \frac{\text{aktive Warps}}{\text{maximal mögliche aktive Warps}}.$$

Die Messwerte liegen in `outputs/occupancy.csv`; die Grafik ist in Abbildung 8 dargestellt.

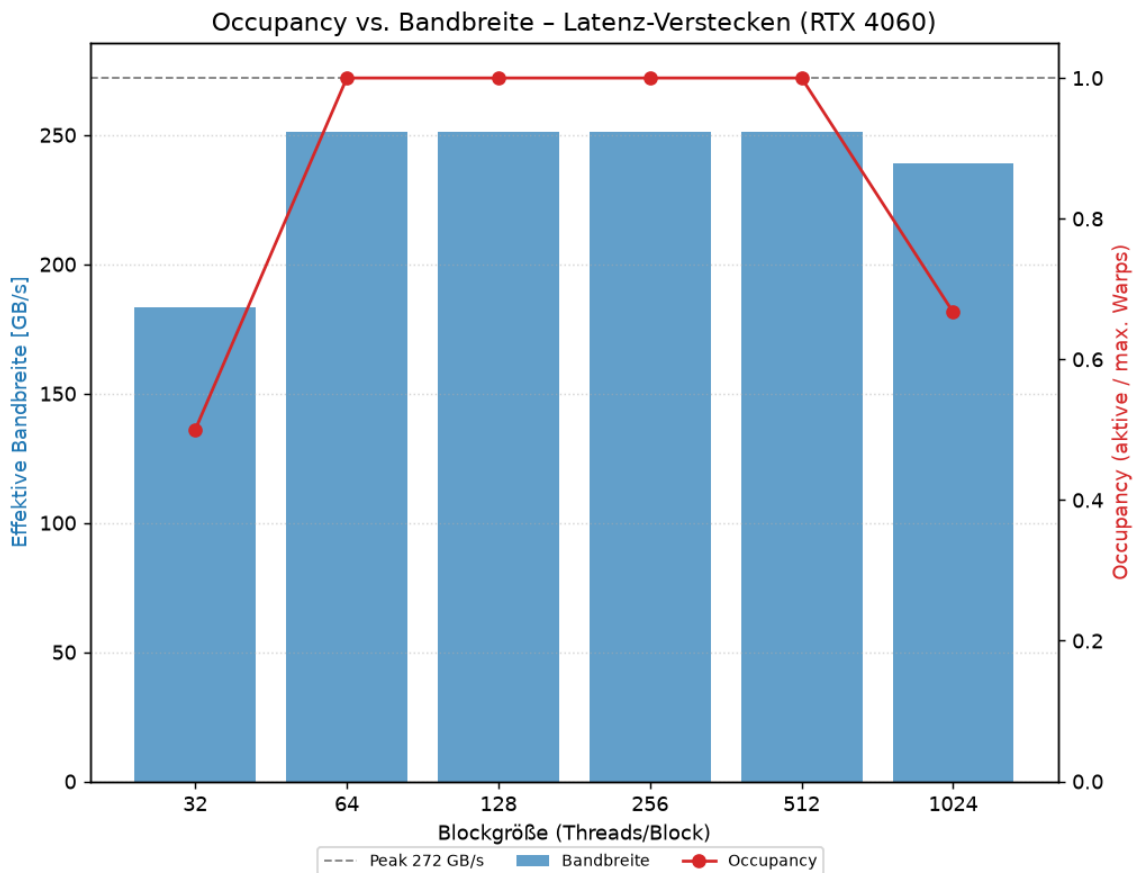


Abbildung 8: Zusammenhang zwischen Blockgröße, *Occupancy* und Bandbreite.

| Blockgröße | Occupancy | GB/s   | % Peak |
|------------|-----------|--------|--------|
| 32         | 0,500     | 183,24 | 67,36  |
| 64         | 1,000     | 251,30 | 92,38  |
| 128        | 1,000     | 251,32 | 92,39  |
| 256        | 1,000     | 251,33 | 92,39  |
| 512        | 1,000     | 251,30 | 92,38  |
| 1024       | 0,667     | 239,14 | 87,91  |

Tabelle 6: *Occupancy-Sweep* für den *coalesced* Kernel.

Bei einer Blockgröße von 32 liegt die *occupancy* nur bei 50% und die Bandbreite fällt auf 183,24 GB/s. Das heißt, es sind zu wenige Warps in einem *Streaming Multiprocessor (SM)*, um Speicher-Stalls vollständig zu überdecken. Ab einer Blockgröße von 64 erreicht die *occupancy* stabil fast 100%. Die Bandbreite steigt dann auf  $\approx 251$  GB/s und bleibt bis zu einer Blockgröße von 512 stabil. Ab einer Blockgröße von 1024 sinkt die *occupancy* wieder auf  $\approx 66,7\%$ , da sich insgesamt weniger große Blöcke gleichzeitig auf einem *Streaming Multiprocessor* befinden können; die Bandbreite sinkt dadurch auf 239,14 GB/s.

Das Experiment zeigt sog. *latency hiding*: Da ein einzelner Speicherzugriff mehrere Zyklen dauern kann, kann der *Warp-Scheduler* bei einem *stall* während der Verarbeitung eines Warps auf einen anderen Warp wechseln, solange sich genügend lauffähige Warps im *Streaming Multiprocessor* befinden. Dadurch wird die Speicherlatenz eines einzelnen Warps zwar nicht vermieden, aber durch Parallelität überdeckt.

### 3.1 GPU- und CPU-Strategien im Vergleich

Die GPU vermeidet lange Speicherlatenzen durch einen hohen Durchsatz, indem sich viele Warps gleichzeitig in den *SMs* befinden und Warp-Scheduling genutzt wird, um lange Speicherlatenzen zu maskieren. Wie Experimente zu den Zugriffsmustern *strided* und *random gather* zeigen, sind GPU Caches im Vergleich zu CPU-Caches klein und können zufällige Zugriffe nicht in gleichem Umfang abfedern. Deshalb entscheidet das Zugriffsmuster direkt über die erreichte Bandbreite.

Die CPU ist dagegen auf eine niedrige Latenz einzelner Instruktionsströme optimiert. Sie nutzt hierfür mehrstufige Cache-Hierarchien, Hardware-Prefetching, Out-of-Order-Execution und wenige starke Kerne. Sequenzielle Zugriffe können dadurch sehr schnell sein, weil Daten rechtzeitig in den Cache geladen werden. Falls zufällige Zugriffe Cache-Misses verursachen, sind diese auch für die CPU teuer.

### 3.2 Folgen für das Datenlayout und GPU-Programme

Aus den Messungen ergeben sich die folgenden Konsequenzen für effiziente GPU-Programme:

- Benachbarte Threads sollten benachbarte Speicheradressen lesen oder schreiben.
- Eine *Structure-of-Arrays* ist für viele GPU-Zugriffsmuster günstiger als ein *Array-of-Structures*, weil Felder dann zusammenhängend gelesen werden.
- *Random Gather* und *Scatter* sollten vermieden oder durch vorheriges Sortieren bzw. Umordnen der Daten reduziert werden.
- Daten sollten passend ausgerichtet und bei 2D-Strukturen gegebenenfalls gepaddet werden.
- Es müssen genug Threads bereitgestellt und passende Blockgrößen gewählt werden, damit sich immer genügend lauffähige Warps gleichzeitig in den *SMs* befinden können.